1 Inertial aiding

When an IMU is configured (`imu.enabled`; the sibling boolean `estimation.aiding.imu` is the `AidingSourceManager` selector and back-compat-defaults to `imu.enabled`, so enabling the legacy flag also enables the aiding-IMU source — an explicit `estimation.aiding.imu` wins, see `src/core/config/GnssConfigLoad.cpp`), the inter-epoch time update of the time update is replaced by a first-order strapdown mechanization in the Earth-centred Earth-fixed (ECEF) frame. The mechanization itself lives in `src/core/filter/ImuIntegrator.cpp`; the live per-epoch predict routes through `Application::run` → `AidingSourceManager::collectPredict` → `src/core/aiding/ImuAidingSource.cpp`, whose `PredictContribution` is spliced into the filter covariance by `KalmanFilter::predictAided` (`src/core/filter/KalmanFilter.cpp`). The real-time engine (`RealTimeEngine::run`) drives the identical `predictAided` / `collectPredict` path; the only difference from the batch run is ingest — a live sensor feeds `ImuIntegrator::addSample`, the streaming counterpart to the CSV `loadData`, which appends to the same sorted sample buffer so streaming and batch yield identical `integrate` results.

This `IAidingSource` / `AidingSourceManager` abstraction (`include/aiding/`) is the composable seam through which every non-GNSS modality — IMU, ZUPT, NHC, baro, wheel odometry, vision — contributes up to three things (a `StateManager`-owned state block, a predict-time Φ/Q sub-block, and either independent measurement rows or extra Jacobian *columns* on existing GNSS rows). The IMU is its first and privileged client; the legacy monolithic `KalmanFilter::predictImu` is retained but no longer on the live path (superseded by `predictAided`). The `ImuAidingSource` replicates `predictImu`'s arithmetic verbatim — the strapdown equations below are unchanged by the refactor.

The formulation is an *error-state* extended Kalman filter. The mechanization propagates the nominal navigation state $(\boldsymbol{r}, \boldsymbol{v}, \boldsymbol{C}_b^e)$ deterministically from the inertial samples, while a 15-element INS error vector

$$\delta\boldsymbol{x}_{\text{ins}} = \begin{bmatrix}\delta\boldsymbol{r} & \delta\boldsymbol{v} & \delta\boldsymbol{\psi} & \delta\boldsymbol{b}_a & \delta\boldsymbol{b}_g\end{bmatrix}^{\mathrm{T}} \in \mathbb{R}^{15} \tag{1}$$

carries the linearised transition and process noise that drive the covariance. Three attitude-error states $\delta\boldsymbol{\psi}$, three accelerometer-bias states $\boldsymbol{b}_a$, and three gyro-bias states $\boldsymbol{b}_g$ augment the existing position state (and, when `estimation.estimate_velocity` is set, the three velocity states); the augmented block is allocated contiguously by `StateManager` (accessors `getAttitudeIndex`, `getAccelBiasIndex`, `getGyroBiasIndex`).

Remark 1 (Implemented scope). The block order of Equation (1) ($[\delta\boldsymbol{r}, \delta\boldsymbol{v}, \delta\boldsymbol{\psi}, \delta\boldsymbol{b}_a, \delta\boldsymbol{b}_g]$) is fixed by `ImuIntegrator`. *No* lever-arm state and *no* sensor scale-factor states are carried, and there is no IMU-to-antenna lever-arm term in the GNSS measurement model (Section 1.4). These are roadmap items, not part of the current implementation; the corresponding terms are omitted from every equation below rather than written and then ignored.

1.1 Strapdown mechanization

For each IMU sample the raw specific force $\tilde{\boldsymbol{a}}$ and angular rate $\tilde{\boldsymbol{\omega}}$ (both in the body frame) are bias-corrected with the current filter estimates,

$$\boldsymbol{f}_b = \tilde{\boldsymbol{a}} - \boldsymbol{b}_a, \qquad \boldsymbol{\omega}_b = \tilde{\boldsymbol{\omega}} - \boldsymbol{b}_g, \tag{2}$$

and integrated over the sample interval Δt. The attitude direction-cosine matrix $\boldsymbol{C}_b^e$ (body $\rightarrow$ ECEF) is updated to first order with Earth-rotation compensation,

$$\boxed{\boldsymbol{C}_b^e \;\leftarrow\; \boldsymbol{C}_b^e\big(\boldsymbol{I} + [\boldsymbol{\omega}_b]_\times\,\Delta t\big) \;-\; [\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{C}_b^e\,\Delta t,} \tag{3}$$

where $\boldsymbol{\omega}_{ie}^e = [\,0,\,0,\,\omega_e\,]^{\mathrm{T}}$ is the ECEF Earth-rotation vector ($\omega_e = 7.292\,115\,146\,7 \times 10^{-5}\,\mathrm{rad\,s^{-1}}$, `common/Constants.hpp`) and $[\cdot]_\times$ is the skew-symmetric (cross-product) matrix. To curb numerical drift, $\boldsymbol{C}_b^e$ is re-orthonormalised by singular-value decomposition every 100 samples. The velocity and position propagate as

$$\boldsymbol{v}_{k+1} = \boldsymbol{v}_k + \big(\boldsymbol{C}_b^e\,\boldsymbol{f}_b + \boldsymbol{g}^e(\boldsymbol{r}_k) - 2\,[\boldsymbol{\omega}_{ie}^e]_\times\,\boldsymbol{v}_k\big)\Delta t, \tag{4}$$

$$\boldsymbol{r}_{k+1} = \boldsymbol{r}_k + \tfrac{1}{2}\big(\boldsymbol{v}_k + \boldsymbol{v}_{k+1}\big)\Delta t. \tag{5}$$

Equation (4) applies the Coriolis acceleration $-2\,[\boldsymbol{\omega}_{ie}^e]_\times\boldsymbol{v}$, while the centrifugal acceleration is folded into the gravity vector $\boldsymbol{g}^e$ (i.e. $\boldsymbol{g}^e$ is a *plumb-bob* gravity, gravitational plus centrifugal); Equation (5) uses a trapezoidal position update.

The gravity vector is a point-mass radial model plus the centrifugal term (`ImuIntegrator::computeGravityEcef`):

$$\boldsymbol{g}^e(\boldsymbol{r}) = -\,\frac{\mu}{\|\boldsymbol{r}\|^3}\,\boldsymbol{r} \;+\; \omega_e^2\,[\,p_x,\,p_y,\,0\,]^{\mathrm{T}}, \qquad \mu = 3.986\,005 \times 10^{14}\,\mathrm{m^3\,s^{-2}}. \tag{6}$$

The normal-gravity (Somigliana) flattening and the J_2 zonal term are *not* modelled; over a single GNSS epoch interval the resulting deterministic acceleration error is small and is absorbed by the estimated accelerometer bias $\boldsymbol{b}_a$.

1.2 INS error-state dynamics

Linearising Equations (3) to (5) about the nominal trajectory yields the continuous error dynamics $\dot{\delta\boldsymbol{x}}_{\mathrm{ins}} = \boldsymbol{F}_{\mathrm{ins}}\,\delta\boldsymbol{x}_{\mathrm{ins}} + \boldsymbol{G}_{\mathrm{ins}}\,\boldsymbol{w}$, with, in the block order of Equation (1),

$$\boldsymbol{F}_{\mathrm{ins}} = \begin{bmatrix} \boldsymbol{0} & \boldsymbol{I} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & -2[\boldsymbol{\omega}_{ie}^e]_\times & -[\boldsymbol{f}^e]_\times & \boldsymbol{C}_b^e & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & -[\boldsymbol{\omega}_{ie}^e]_\times & \boldsymbol{0} & -\boldsymbol{C}_b^e \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} & \boldsymbol{0} \end{bmatrix}, \qquad \boldsymbol{f}^e := \boldsymbol{C}_b^e\,\boldsymbol{f}_b. \tag{7}$$

Both sensor biases are modelled as pure random walks (zero deterministic dynamics, the null bottom rows of Equation (7)). Two terms of the textbook ECEF error model are deliberately dropped: the position-error feedback row ($[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{r},\delta\boldsymbol{r}} = -[\boldsymbol{\omega}_{ie}^e]_\times$) and the gravity-gradient coupling $[\boldsymbol{F}_{\mathrm{ins}}]_{\delta\boldsymbol{v},\delta\boldsymbol{r}} = +\,\partial\boldsymbol{g}^e/\partial\boldsymbol{r}$ (the gradient of the *plumb* gravity of Equation (6), which already subsumes the centrifugal $-[\boldsymbol{\omega}_{ie}^e]_\times^2$; the leading sign follows from differentiating the velocity update Equation (4)). This is a first-order approximation that is accurate over the short epoch interval of a GNSS-aided run. The discrete transition is taken to first order,

$$[\boldsymbol{\Phi}_{\mathrm{ins}}]_k = \boldsymbol{I} + \boldsymbol{F}_{\mathrm{ins}}\,\Delta t. \tag{8}$$

The process noise maps the continuous accelerometer / gyro white noise and the bias random walks through

$$[\boldsymbol{Q}_{\text{ins}}]_k = \boldsymbol{G}_{\text{ins}}\,\boldsymbol{W}\,\boldsymbol{G}_{\text{ins}}^{\mathrm{T}}\,\Delta t, \quad \boldsymbol{G}_{\text{ins}} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \boldsymbol{C}_b^e & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & -\boldsymbol{C}_b^e & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \boldsymbol{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \boldsymbol{I} \end{bmatrix}, \quad \boldsymbol{W} = \operatorname{diag}\!\big(\sigma_a^2, \sigma_g^2, \sigma_{ba}^2, \sigma_{bg}^2\big) \otimes \boldsymbol{I}_3, \tag{9}$$

with the spectral densities $\sigma_a, \sigma_g, \sigma_{ba}, \sigma_{bg}$ read from `imu.{accel_noise_density, gyro_noise_density, accel_bias_instability, gyro_bias_instability}` (defaults in `config/GnssConfigTypes.hpp`). Over one GNSS epoch the per-sample $([\boldsymbol{\Phi}_{\text{ins}}]_k, [\boldsymbol{Q}_{\text{ins}}]_k)$ accumulate by the standard recursion

$$\boldsymbol{\Phi}_{\text{ins}} \leftarrow [\boldsymbol{\Phi}_{\text{ins}}]_k\,\boldsymbol{\Phi}_{\text{ins}}, \qquad \boldsymbol{Q}_{\text{ins}} \leftarrow [\boldsymbol{\Phi}_{\text{ins}}]_k\,\boldsymbol{Q}_{\text{ins}}\,[\boldsymbol{\Phi}_{\text{ins}}]_k^{\mathrm{T}} + [\boldsymbol{Q}_{\text{ins}}]_k. \tag{10}$$

1.3 Covariance splice and attitude feedback

The accumulated 15×15 pair $(\boldsymbol{\Phi}_{\text{ins}}, \boldsymbol{Q}_{\text{ins}})$ is spliced (*assigned*, not accumulated) onto an identity-seeded full-state transition $\boldsymbol{F}$ and onto the random-walk process-noise baseline $\boldsymbol{Q}$ over the indices the INS owns; the non-INS states (clock, troposphere, ionosphere, ambiguities, ...) retain their standard random-walk model from the time update. The covariance is then propagated once and symmetrised,

$$\boxed{\boldsymbol{P} \leftarrow \boldsymbol{F}\,\boldsymbol{P}\,\boldsymbol{F}^{\mathrm{T}} + \boldsymbol{Q}, \qquad \boldsymbol{P} \leftarrow \tfrac{1}{2}\big(\boldsymbol{P} + \boldsymbol{P}^{\mathrm{T}}\big).} \tag{11}$$

The nominal navigation states $(\boldsymbol{r}, \boldsymbol{v})$ are overwritten by the mechanised values of Equations (4) to (5), and the attitude error is reset, $\delta\boldsymbol{\psi} := \mathbf{0}$, because the rotation increment has already been applied directly to $\boldsymbol{C}_b^e$ in Equation (3). After the subsequent GNSS measurement update the corrected attitude error feeds back multiplicatively into the DCM and is again reset,

$$\boldsymbol{C}_b^e \leftarrow \big(\boldsymbol{I} + [\delta\boldsymbol{\psi}]_\times\big)\boldsymbol{C}_b^e, \qquad \delta\boldsymbol{\psi} := \mathbf{0}, \tag{12}$$

followed by SVD re-orthonormalisation (`KalmanFilter::update`). This closed-loop reset keeps $\delta\boldsymbol{\psi}$ a small-angle quantity throughout, so the linearisation in Equation (7) stays valid.

1.4 Coupling modes and the GNSS update

How the GNSS observations correct the inertial errors depends on the coupling mode (`imu.coupling_mode`).

Loosely coupled (default). The GNSS code and carrier-phase Jacobians of the observation geometry touch only the position (and velocity) columns. The INS error states $\delta\boldsymbol{\psi}, \boldsymbol{b}_a, \boldsymbol{b}_g$ therefore receive *no direct* observation partials; they are corrected only indirectly, through the off-diagonal cross-covariances that Equation (11) builds between the position/velocity block and the attitude/bias block. The attitude correction is then realised by the feedback of Equation (12).

Tightly coupled. Tightly-coupled mode requires the velocity block, so the loader force-enables it independently of the user's `estimation.estimate_velocity` — a `TightCoupled coupling_mode` with the flag left false flips it true and logs `"forcing estimate_velocity=true"` (`GnssConfigLoad.cpp`), guaranteeing the three velocity states and hence the Doppler rows the TC Jacobian below

attaches to. In this mode the *Doppler* observation alone gains direct attitude and accelerometer-bias columns. This is the one aiding contribution that re-parameterizes existing GNSS rows rather than appending its own: each relinearisation pass calls `AidingSourceManager::applyTcJacobianInputs` → `ImuAidingSource::applyTcJacobianInputs`, which feeds the current attitude and specific force to `MeasurementModel::setImuState` (`src/core/measurement/MeasurementModel.cpp`). With $\boldsymbol{f}^e = \boldsymbol{C}_b^e \boldsymbol{f}_b$ the specific force in ECEF and $\boldsymbol{e}_r^i$ the unit line-of-sight of the line-of-sight definition, the Doppler measurement function h_{D} contributes

$$\boxed{\frac{\partial h_{\mathrm{D}}}{\partial \delta \boldsymbol{\psi}} = -\left[\boldsymbol{f}^e\right]_{\times}^{\mathrm{T}} \boldsymbol{e}_r^i, \qquad \frac{\partial h_{\mathrm{D}}}{\partial \boldsymbol{b}_a} = -\left(\boldsymbol{C}_b^e\right)^{\mathrm{T}} \boldsymbol{e}_r^i.} \tag{13}$$

The specific force fed into Equation (13) is a static-rest approximation $\boldsymbol{f}_b \approx (\boldsymbol{C}_b^e)^{\mathrm{T}}\left(-\boldsymbol{g}^e\right) - \boldsymbol{b}_a$ (`ImuAidingSource.cpp`); it is exact only when the platform's true acceleration is negligible against gravity. The *code and carrier-phase rows carry no attitude or bias partials in either mode* — only Doppler couples directly to the inertial errors.

Remark 2 (Relinearisation under tight coupling). The tightly-coupled path retains the legacy single *naive* second-pass relinearisation rather than the prior-anchored iterated EKF of the iterated EKF: re-deriving the gain from a fixed prior conflicts with the multiplicative attitude/bias feedback of Equation (12), which has already mutated $\boldsymbol{C}_b^e$ between passes. The aiding layer exposes this as a capability query (`AidingSourceManager::anySourceRequiresNaiveRelinearization`, which the IMU source answers true for in tight coupling), so any future nominal-feedback source forces the same path; the migration off the direct `tightlyCoupledImu()` checks in `Application.cpp` / `KalmanFilter.cpp` is still in progress.